> 🛠️ **2026.10 수정본 (v11)** — 원본: ch02_03_02B_02_Ticket_gender.ipynb
>
> train·test를 합칠 때 인덱스 번호가 겹쳐서(0~890, 0~417) pandas 3에서 `pd.crosstab`이 에러를 냈어요. `pd.concat(..., ignore_index=True)`로 번호를 새로 매기도록 한 줄만 바꿨습니다.

타이타닉 생존자 예측 대회

학습 목표

- Ticket, Gender 피처를 사용한다.
- GridSearchCV를 통해 좋은 변수를 사용한다.
- 데이터 변환을 수행해 본다.
- 새로운 변수 생성을 알아본다.

목차

01. 데이터 불러오기
02. 데이터 전처리
03. 모델링
04. 예측

데이터

Data Fields

구분 | 설명 | 값 | -----|-----|-----| **Survival** | 생존 여부 | Survival. 0 = No, 1 = Yes **Pclass** | 티켓의 클래스 |Ticket class. 1 = 1st, 2 = 2nd, 3 = 3rd **Sex** | 성별(Sex) | 남(male)/여(female) **Age** | 나이(Age in years.) | **SibSp** | 함께 탑승한 형제와 배우자의 수 /siblings, spouses aboard the Titanic. | **Parch** | 함께 탑승한 부모, 아이의 수 | # of parents / children aboard the Titanic. **Ticket** | 티켓 번호(Ticket number) | (ex) CA 31352, A/5. 2151 **Fare** | 탑승료(Passenger fare) | **Cabin** | 객실 번호(Cabin number) | **Embarked** | 탑승 항구(Port of Embarkation) | C = Cherbourg, Q = Queenstown, S = Southampton

- siblings : 형제, 자매, 형제, 의붓 형제
- spouses : 남편, 아내 (정부와 약혼자는 무시)
- Parch : Parent(mother, father), child(daughter, son, stepdaughter, stepson)

01. 데이터 불러오기

목차로 이동하기

참고 노트북

- titanic 전체 노트북
  - https://www.kaggle.com/code/pliptor/how-am-i-doing-with-my-score/report
  - https://www.kaggle.com/code/pliptor/titanic-ticket-only-study/notebook

In [1]:
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
```

In [2]:
```
sel_f = ['Ticket', 'Pclass', 'Sex']

train = pd.read_csv("data/titanic/train.csv",
                    usecols=['PassengerId', 'Survived']+sel_f)

test = pd.read_csv("data/titanic/test.csv",
                   usecols=['PassengerId'] + sel_f)

sub = pd.read_csv("data/titanic/gender_submission.csv")
```

In [3]:
```
# 컬럼 추가 및 합치기
test['Survived'] = np.nan
all_df = pd.concat([train, test], ignore_index=True)  # 2026.10 수정: 인덱스 중
all_df.head()
```

Out[3]:

	PassengerId	Survived	Pclass	Sex	Ticket
0	1	0.0	3	male	A/5 21171
1	2	1.0	1	female	PC 17599
2	3	1.0	3	female	STON/O2. 3101282
3	4	1.0	1	female	113803
4	5	0.0	3	male	373450

02. 티켓 변수 확인

목차로 이동하기

In [4]:
```
all_df.Ticket.unique()
```

Out[4]:
```
<StringArray>
[         'A/5 21171',           'PC 17599',   'STON/O2. 3101282',
             '113803',             '373450',             '330877',
              '17463',             '349909',             '347742',
             '237736',
 ...
              '21332',              '17765',      'SC/PARIS 2166',
              '28666',             '334915',             '365237',
             '347086',          'A.5. 3236', 'SOTON/O.Q. 3101262',
             '359309']
Length: 929, dtype: str
```

- 티켓은 순전히 숫자인 것, 그리고 영숫자 접두사, 그리고 승무원인 LINE이 발행.

In [5]:
```
all_df.loc[all_df['Ticket']=='LINE']
```

Out[5]:

	PassengerId	Survived	Pclass	Sex	Ticket
179	180	0.0	3	male	LINE
271	272	1.0	3	male	LINE
302	303	0.0	3	male	LINE
597	598	0.0	3	male	LINE

- 다른 데이터와 비슷하게 만들기 위해 LINE 0 으로 변경

In [6]:
```
all_df['Ticket'] = all_df['Ticket'].replace('LINE', 'LINE 0')
all_df[all_df['Ticket']=='LINE 0']
```

Out[6]:

	PassengerId	Survived	Pclass	Sex	Ticket
179	180	0.0	3	male	LINE 0
271	272	1.0	3	male	LINE 0
302	303	0.0	3	male	LINE 0
597	598	0.0	3	male	LINE 0

티켓의 중복 확인

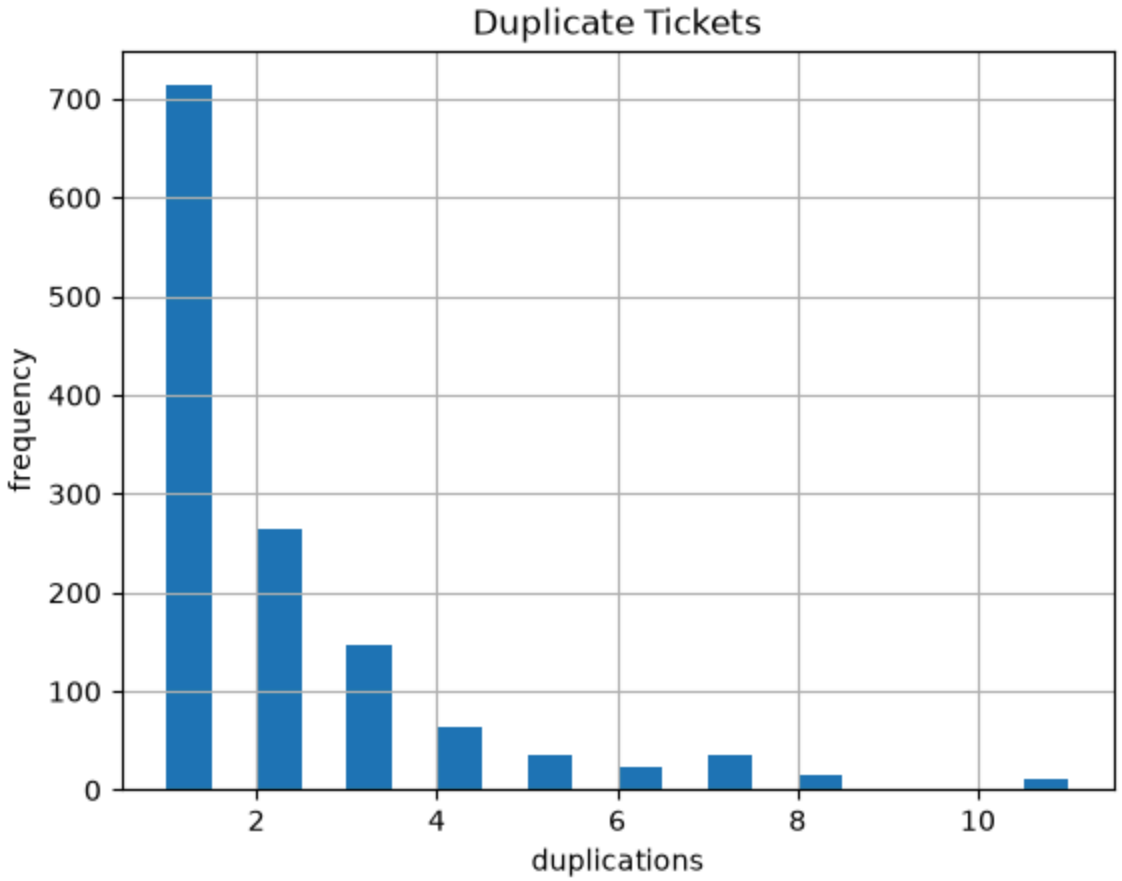
In [7]:
```
dup_tickets = all_df.groupby('Ticket').size()
dup_tickets
```

Out[7]:
```
Ticket
110152         3
110413         3
110465         2
110469         1
110489         1
              ..
W./C. 6608     5
W./C. 6609     1
W.E.P. 5734    2
W/C 14208      1
WE/P 5735      2
Length: 929, dtype: int64
```

중복 티켓 장수

In [8]:
```
all_df['중복티켓수'] = all_df['Ticket'].map(dup_tickets)
plt.xlabel('duplications')
plt.ylabel('frequency')
plt.title('Duplicate Tickets')
all_df['중복티켓수'].hist(bins=20)
```

Out[8]:
```
<Axes: title={'center': 'Duplicate Tickets'}, xlabel='duplications', ylabel
='frequency'>
```



- 유일한 티켓이 압도적으로 많다.

티켓의 값 처리

- '.', '/' 을 공백으로 변경

In [9]:
```
all_df['Ticket'] = all_df['Ticket'].apply(lambda x: x.replace('.','').replac
all_df.head()
```

Out[9]:

	PassengerId	Survived	Pclass	Sex	Ticket	중복티켓수
0	1	0.0	3	male	a5 21171	1
1	2	1.0	1	female	pc 17599	2
2	3	1.0	3	female	stono2 3101282	1
3	4	1.0	1	female	113803	2
4	5	0.0	3	male	373450	1

Ticket의 값을 공백으로 분리 후, 앞의 이름을 갖는 변수 만들기

In [10]:
```
"aaaaa 000000".split(' ')[0][0]     # 첫번째 단어의 맨 앞 첫글자
```

Out[10]:
```
'a'
```

In [11]:
```
def get_prefix(ticket):
    lead = ticket.split(' ')[0][0]

    # 알파벳인지 확인
    if lead.isalpha():
        return ticket.split(' ')[0]
    else:
        return 'NoPrefix'

all_df['Prefix'] = all_df['Ticket'].apply(lambda x: get_prefix(x))
all_df.head()
```

Out[11]:

	PassengerId	Survived	Pclass	Sex	Ticket	중복티켓수	Prefix
0	1	0.0	3	male	a5 21171	1	a5
1	2	1.0	1	female	pc 17599	2	pc
2	3	1.0	3	female	stono2 3101282	1	stono2
3	4	1.0	1	female	113803	2	NoPrefix
4	5	0.0	3	male	373450	1	NoPrefix

In [12]:
```
"a5 21171".split(' ')[-1]
```

Out[12]:
```
'21171'
```

In [13]:
```
str("a5 21171")[0]
```

Out[13]:
```
'a'
```

In [14]:
```
val = int( "a5 21171".split(' ')[-1] )
str(val)
```

Out[14]:
```
'21171'
```

- TNumeric : 숫자로 변경
- TNlen : TNumeric의 길이
- LeadingDigit : TNumeric의 맨 앞글자
- TGroup : Ticket의 뒷부분의 문자로 변경

In [15]:
```
all_df['TNumeric'] = all_df['Ticket'].apply(lambda x: int(x.split(' ')[-1])/
all_df['TNlen'] = all_df['TNumeric'].apply(lambda x : len(str(x)))
all_df['LeadingDigit'] = all_df['TNumeric'].apply(lambda x : int(str(x)[0]))
all_df['TGroup'] = all_df['Ticket'].apply(lambda x: str(int(x.split(' ')[-1]
all_df.head()
```

Out[15]:

	PassengerId	Survived	Pclass	Sex	Ticket	중복티켓수	Prefix	TNumeric	TNlen	Le
0	1	0.0	3	male	a5 21171	1	a5	21171	5	
1	2	1.0	1	female	pc 17599	2	pc	17599	5	
2	3	1.0	3	female	stono2 3101282	1	stono2	3101282	7	
3	4	1.0	1	female	113803	2	NoPrefix	113803	6	
4	5	0.0	3	male	373450	1	NoPrefix	373450	6	

In [16]:
```
pd.crosstab(all_df['Pclass'],all_df['LeadingDigit'])
```

Out[16]:

LeadingDigit	0	1	2	3	4	5	6	7	8	9
Pclass										
1	0	288	8	18	0	5	4	0	0	0
2	0	32	205	37	0	1	0	2	0	0
3	4	22	136	476	22	4	17	18	5	5

In [17]:
```
all_df = all_df.drop(columns=['Ticket', 'TNumeric', 'Pclass'])
all_df
```

Out[17]:

	PassengerId	Survived	Sex	중복티켓수	Prefix	TNlen	LeadingDigit	TGroup
0	1	0.0	male	1	a5	5	2	2117
1	2	1.0	female	2	pc	5	1	1759
2	3	1.0	female	1	stono2	7	3	310128
3	4	1.0	female	2	NoPrefix	6	1	11380
4	5	0.0	male	1	NoPrefix	6	3	37345
...	...	...	...	...	...	...	...	...
1304	1305	NaN	male	1	a5	4	3	323
1305	1306	NaN	female	3	pc	5	1	1775
1306	1307	NaN	male	1	sotonoq	7	3	310126
1307	1308	NaN	male	1	NoPrefix	6	3	35930
1308	1309	NaN	male	3	NoPrefix	4	2	266

1309 rows × 8 columns

In [18]:
```
all_df['Prefix']
```

Out[18]:
```
0            a5
1            pc
2        stono2
3      NoPrefix
4      NoPrefix
         ...
1304         a5
1305         pc
1306    sotonoq
1307   NoPrefix
1308   NoPrefix
Name: Prefix, Length: 1309, dtype: str
```

In [19]:
```
all_df = pd.concat([pd.get_dummies(all_df[['Prefix','TGroup']]),
            all_df[['PassengerId','Survived','중복티켓수','TNlen','LeadingDigit
            axis=1)

all_df
```

Out[19]:

	Prefix_NoPrefix	Prefix_a	Prefix_a4	Prefix_a5	Prefix_aq3	Prefix_aq4	Prefix_
0	False	False	False	True	False	False	Fa
1	False	False	False	False	False	False	Fa
2	False	False	False	False	False	False	Fa
3	True	False	False	False	False	False	Fa
4	True	False	False	False	False	False	Fa
...	...	...	...	...	...	...	
1304	False	False	False	True	False	False	Fa
1305	False	False	False	False	False	False	Fa
1306	False	False	False	False	False	False	Fa
1307	True	False	False	False	False	False	Fa
1308	True	False	False	False	False	False	Fa

1309 rows × 451 columns

In [20]:
```
dict_s = {'male':0, 'female':1}
all_df['Sex'] = all_df['Sex'].map(dict_s)
```

In [21]:
```
predictors = sorted(list(set(all_df.columns) - set(['PassengerId','Survived'
predictors
```

Out[21]:
```
['LeadingDigit',
 'Prefix_NoPrefix',
 'Prefix_a',
 'Prefix_a4',
 'Prefix_a5',
 'Prefix_aq3',
 'Prefix_aq4',
 'Prefix_as',
 'Prefix_c',
 'Prefix_ca',
 'Prefix_casoton',
 'Prefix_fa',
 'Prefix_fc',
 'Prefix_fcc',
 'Prefix_line',
 'Prefix_lp',
 'Prefix_pc',
 'Prefix_pp',
 'Prefix_ppp',
 'Prefix_sc',
 'Prefix_sca3',
 'Prefix_sca4',
 'Prefix_scah',
 'Prefix_scow',
 'Prefix_scparis',
 'Prefix_soc',
 'Prefix_sop',
 'Prefix_sopp',
 'Prefix_sotono2',
 'Prefix_sotonoq',
 'Prefix_sp',
 'Prefix_stono',
 'Prefix_stono2',
 'Prefix_stonoq',
 'Prefix_swpp',
 'Prefix_wc',
 'Prefix_wep',
 'Sex',
 'TGroup_0',
 'TGroup_1048',
 'TGroup_11015',
 'TGroup_11041',
 'TGroup_11046',
 'TGroup_11048',
 'TGroup_11056',
 'TGroup_11081',
 'TGroup_11116',
 'TGroup_11124',
 'TGroup_11132',
 'TGroup_11136',
 'TGroup_11142',
 'TGroup_1120',
 'TGroup_11205',
 'TGroup_11227',
 'TGroup_11237',
 'TGroup_11290',
```

'TGroup_11302',
'TGroup_11303',
'TGroup_11304',
'TGroup_11305',
'TGroup_11350',
'TGroup_11351',
'TGroup_11357',
'TGroup_11376',
'TGroup_11377',
'TGroup_11378',
'TGroup_11379',
'TGroup_11380',
'TGroup_116',
'TGroup_1166',
'TGroup_1175',
'TGroup_1176',
'TGroup_1177',
'TGroup_1181',
'TGroup_1196',
'TGroup_122',
'TGroup_1223',
'TGroup_1246',
'TGroup_1274',
'TGroup_1275',
'TGroup_1299',
'TGroup_1303',
'TGroup_1304',
'TGroup_1305',
'TGroup_1321',
'TGroup_1323',
'TGroup_1350',
'TGroup_1352',
'TGroup_1353',
'TGroup_1354',
'TGroup_1356',
'TGroup_1369',
'TGroup_1390',
'TGroup_1420',
'TGroup_1425',
'TGroup_1426',
'TGroup_1431',
'TGroup_147',
'TGroup_1487',
'TGroup_1488',
'TGroup_1497',
'TGroup_1518',
'TGroup_158',
'TGroup_160',
'TGroup_1696',
'TGroup_1698',
'TGroup_1724',
'TGroup_1731',
'TGroup_1736',
'TGroup_174',
'TGroup_1742',
'TGroup_1745',

```
'TGroup_1746',
'TGroup_1747',
'TGroup_1748',
'TGroup_1753',
'TGroup_1755',
'TGroup_1756',
'TGroup_1757',
'TGroup_1758',
'TGroup_1759',
'TGroup_1760',
'TGroup_1761',
'TGroup_1775',
'TGroup_1776',
'TGroup_1777',
'TGroup_1850',
'TGroup_1872',
'TGroup_1987',
'TGroup_1992',
'TGroup_1994',
'TGroup_1995',
'TGroup_1997',
'TGroup_1998',
'TGroup_1999',
'TGroup_200',
'TGroup_2058',
'TGroup_207',
'TGroup_21153',
'TGroup_2117',
'TGroup_212',
'TGroup_2122',
'TGroup_213',
'TGroup_2133',
'TGroup_214',
'TGroup_2144',
'TGroup_215',
'TGroup_216',
'TGroup_21862',
'TGroup_21953',
'TGroup_22036',
'TGroup_22084',
'TGroup_222',
'TGroup_22359',
'TGroup_22659',
'TGroup_22687',
'TGroup_22841',
'TGroup_22923',
'TGroup_23008',
'TGroup_23013',
'TGroup_23043',
'TGroup_231',
'TGroup_23191',
'TGroup_23194',
'TGroup_23347',
'TGroup_23363',
'TGroup_23373',
'TGroup_23386',
```

'TGroup_234',
'TGroup_23436',
'TGroup_23460',
'TGroup_23468',
'TGroup_23481',
'TGroup_23550',
'TGroup_2356',
'TGroup_23617',
'TGroup_23685',
'TGroup_23721',
'TGroup_23724',
'TGroup_23739',
'TGroup_23744',
'TGroup_23756',
'TGroup_23766',
'TGroup_23767',
'TGroup_23773',
'TGroup_23778',
'TGroup_23779',
'TGroup_23905',
'TGroup_23985',
'TGroup_23986',
'TGroup_24026',
'TGroup_24027',
'TGroup_2406',
'TGroup_24092',
'TGroup_2416',
'TGroup_24296',
'TGroup_24384',
'TGroup_24388',
'TGroup_24425',
'TGroup_24427',
'TGroup_24431',
'TGroup_24434',
'TGroup_24435',
'TGroup_24436',
'TGroup_24437',
'TGroup_2457',
'TGroup_2458',
'TGroup_246',
'TGroup_24865',
'TGroup_24869',
'TGroup_24870',
'TGroup_24872',
'TGroup_24873',
'TGroup_24874',
'TGroup_25',
'TGroup_25064',
'TGroup_25065',
'TGroup_254',
'TGroup_262',
'TGroup_263',
'TGroup_2636',
'TGroup_264',
'TGroup_265',
'TGroup_26530',

'TGroup_266',
'TGroup_267',
'TGroup_2670',
'TGroup_268',
'TGroup_269',
'TGroup_270',
'TGroup_2704',
'TGroup_2726',
'TGroup_2784',
'TGroup_2800',
'TGroup_2803',
'TGroup_281',
'TGroup_2813',
'TGroup_2820',
'TGroup_2821',
'TGroup_2822',
'TGroup_2840',
'TGroup_2842',
'TGroup_2855',
'TGroup_286',
'TGroup_2866',
'TGroup_290',
'TGroup_2901',
'TGroup_2903',
'TGroup_2910',
'TGroup_2917',
'TGroup_292',
'TGroup_2939',
'TGroup_2956',
'TGroup_2975',
'TGroup_3063',
'TGroup_3076',
'TGroup_308',
'TGroup_310126',
'TGroup_310127',
'TGroup_310128',
'TGroup_310129',
'TGroup_310130',
'TGroup_310131',
'TGroup_3102',
'TGroup_3103',
'TGroup_31299',
'TGroup_313',
'TGroup_3135',
'TGroup_3141',
'TGroup_31503',
'TGroup_31508',
'TGroup_31509',
'TGroup_31515',
'TGroup_3192',
'TGroup_323',
'TGroup_3230',
'TGroup_32359',
'TGroup_32395',
'TGroup_32466',
'TGroup_32994',

'TGroup_33084',
'TGroup_33087',
'TGroup_33090',
'TGroup_33091',
'TGroup_33092',
'TGroup_33093',
'TGroup_33095',
'TGroup_33096',
'TGroup_33097',
'TGroup_33098',
'TGroup_3311',
'TGroup_333',
'TGroup_33491',
'TGroup_33509',
'TGroup_33543',
'TGroup_33567',
'TGroup_3359',
'TGroup_3363',
'TGroup_33643',
'TGroup_338',
'TGroup_3405',
'TGroup_3406',
'TGroup_341',
'TGroup_34182',
'TGroup_3421',
'TGroup_3424',
'TGroup_34244',
'TGroup_3426',
'TGroup_34268',
'TGroup_34271',
'TGroup_34282',
'TGroup_34309',
'TGroup_34312',
'TGroup_34327',
'TGroup_34536',
'TGroup_34549',
'TGroup_34550',
'TGroup_34557',
'TGroup_34576',
'TGroup_34577',
'TGroup_34578',
'TGroup_346',
'TGroup_3464',
'TGroup_3465',
'TGroup_347',
'TGroup_34705',
'TGroup_34706',
'TGroup_34707',
'TGroup_34708',
'TGroup_34709',
'TGroup_34746',
'TGroup_34747',
'TGroup_34774',
'TGroup_34812',
'TGroup_34920',
'TGroup_34921',

```
'TGroup_34922',
'TGroup_34923',
'TGroup_34924',
'TGroup_34925',
'TGroup_34990',
'TGroup_34991',
'TGroup_35002',
'TGroup_35003',
'TGroup_35004',
'TGroup_35005',
'TGroup_35006',
'TGroup_35040',
'TGroup_35041',
'TGroup_3527',
'TGroup_3528',
'TGroup_353',
'TGroup_354',
'TGroup_3585',
'TGroup_35858',
'TGroup_359',
'TGroup_35930',
'TGroup_3620',
'TGroup_36231',
'TGroup_36327',
'TGroup_36329',
'TGroup_36359',
'TGroup_36361',
'TGroup_36449',
'TGroup_36450',
'TGroup_36451',
'TGroup_36484',
'TGroup_36485',
'TGroup_36522',
'TGroup_36523',
'TGroup_3656',
'TGroup_36671',
'TGroup_36722',
'TGroup_36723',
'TGroup_36765',
'TGroup_36832',
'TGroup_36836',
'TGroup_36840',
'TGroup_36857',
'TGroup_3686',
'TGroup_36870',
'TGroup_36878',
'TGroup_3692',
'TGroup_3694',
'TGroup_3696',
'TGroup_3697',
'TGroup_36994',
'TGroup_370',
'TGroup_37012',
'TGroup_37036',
'TGroup_37037',
'TGroup_37106',
```

'TGroup_37110',
'TGroup_37111',
'TGroup_37136',
'TGroup_37262',
'TGroup_37345',
'TGroup_37474',
'TGroup_37488',
'TGroup_37491',
'TGroup_37656',
'TGroup_3767',
'TGroup_38264',
'TGroup_38265',
'TGroup_38312',
'TGroup_38316',
'TGroup_38446',
'TGroup_38652',
'TGroup_390',
'TGroup_3918',
'TGroup_39207',
'TGroup_39208',
'TGroup_39209',
'TGroup_39414',
'TGroup_3988',
'TGroup_400',
'TGroup_413',
'TGroup_4279',
'TGroup_434',
'TGroup_4538',
'TGroup_457',
'TGroup_4887',
'TGroup_4986',
'TGroup_54',
'TGroup_5451',
'TGroup_5463',
'TGroup_554',
'TGroup_572',
'TGroup_573',
'TGroup_621',
'TGroup_6530',
'TGroup_656',
'TGroup_660',
'TGroup_68',
'TGroup_69',
'TGroup_707',
'TGroup_726',
'TGroup_75',
'TGroup_753',
'TGroup_754',
'TGroup_755',
'TGroup_759',
'TGroup_793',
'TGroup_847',
'TGroup_85',
'TGroup_923',
'TGroup_954',

```
 'TNlen',
 '중복티켓수']
```

In [22]:
```
all_df2 = all_df[predictors + ['Survived']]
all_df2.head()
```

Out[22]:

	LeadingDigit	Prefix_NoPrefix	Prefix_a	Prefix_a4	Prefix_a5	Prefix_aq3	Prefix_ac
0	2	False	False	False	True	False	Fal
1	1	False	False	False	False	False	Fal
2	3	False	False	False	False	False	Fal
3	1	True	False	False	False	False	Fal
4	3	True	False	False	False	False	Fal

5 rows × 450 columns

In [23]:
```
df_train = all_df2.loc[all_df2['Survived'].isin([np.nan]) == False]
df_test  = all_df2.loc[all_df2['Survived'].isin([np.nan]) == True]

print(df_train.shape)
df_train.head()
```

```
(891, 450)
```

Out[23]:

	LeadingDigit	Prefix_NoPrefix	Prefix_a	Prefix_a4	Prefix_a5	Prefix_aq3	Prefix_ac
0	2	False	False	False	True	False	Fal
1	1	False	False	False	False	False	Fal
2	3	False	False	False	False	False	Fal
3	1	True	False	False	False	False	Fal
4	3	True	False	False	False	False	Fal

5 rows × 450 columns

In [24]:
```
print(df_test.shape)
df_test.head()
```

```
(418, 450)
```

Out[24]:

	LeadingDigit	Prefix_NoPrefix	Prefix_a	Prefix_a4	Prefix_a5	Prefix_aq3	Prefix_
891	3	True	False	False	False	False	
892	3	True	False	False	False	False	
893	2	True	False	False	False	False	
894	3	True	False	False	False	False	
895	3	True	False	False	False	False	

5 rows × 450 columns

03. 모델링

목차로 이동하기

In [25]:
```
from sklearn.model_selection import GridSearchCV
from sklearn.neighbors import KNeighborsClassifier
```

In [26]:
```
model = KNeighborsClassifier(n_neighbors=11, metric = 'manhattan')

param_grid = ({'n_neighbors':[6,7,8,9,11],
               'metric':['manhattan','minkowski'],
               'p':[1,2]})

grs = GridSearchCV(model, param_grid,
                   cv = 28,
                   n_jobs=1,
                   return_train_score = True,
                   pre_dispatch=1)

grs.fit(np.array(df_train[predictors]), np.array(df_train['Survived']))
```

Out[26]:

▸ **GridSearchCV** ⓘ ?

▸ **best_estimator_: KNeighborsClassifier**

▸ KNeighborsClassifier ?

In [27]:
```
print("Best parameters " + str(grs.best_params_))
gpd = pd.DataFrame(grs.cv_results_)
print("정확도 :{0:1.4f}".format(gpd['mean_test_score'][grs.best_index_]))
```

```
Best parameters {'metric': 'manhattan', 'n_neighbors': 11, 'p': 1}
정확도 :0.7846
```

04. 예측

목차로 이동하기

In [28]:
```
pred_knn = grs.predict(np.array(df_test[predictors]))

sub = pd.DataFrame({'PassengerId':test['PassengerId'],'Survived':pred_knn})
sub.to_csv('ticket__sex_knn.csv', index = False, float_format='%1d')
sub.head()
```

Out[28]:

	PassengerId	Survived
0	892	0.0
1	893	1.0
2	894	0.0
3	895	0.0
4	896	0.0

0.72009

실습

- 1. 다른 feature도 추가한 이후에 제출해 보기
  - 'PassengerId', 'Pclass', 'SibSp', 'Parch'
- 2. Age, Fare, Embarked를 추가한 이후에 제출해 보기

In [ ]:
```
```